Week 8 Level 2: Intermediate

Testing: Prove It Works

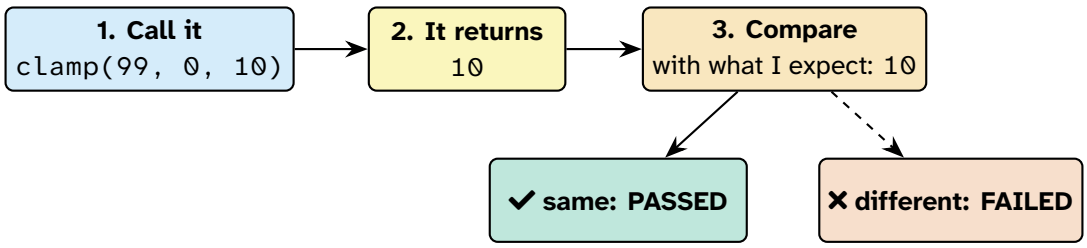
A test is a tiny program that checks your program, so bugs get caught before players find them.

What you will learn

- How to write a **test** with `assert`.
- How to run your tests with **pytest** and read PASSED and FAILED.
- How to choose **normal** and **edge** test cases that catch sneaky bugs.
- How to use failing tests as a map to find and fix bugs.

The big idea: let the computer check your code

Checking a game by hand is slow: run it, play, look at the score... again and again. A **test** is a tiny function that checks another function for you, in less than a second. It uses the word `assert`, which means "I am sure this is true".



A test for clamp(), which keeps a number between two limits

```
from clamp_tools import clamp

def test_too_big_comes_down_to_high():
    assert clamp(99, 0, 10) == 10, "99 is too big, so it should become 10"
```

A test has three parts: **call** the function with values you choose, say what you **expect** with ==, and add a **message** for when it goes wrong. The function's name must start with `test_`, so that pytest can find it.

Running your tests

- **Easy way:** open your test file and press **Run**. The bottom of the file starts pytest for you.
- **Programmer way: Tools > Open system shell...**, type `python -m pytest starter.py -v` and press `Return` (if it says "command not found", use `python3`).

Output

```
starter.py::test_middle_value_stays_the_same PASSED     [ 50%]
starter.py::test_too_small_goes_up_to_low FAILED        [100%]
E       AssertionError: -3 is too small, so it should become 0
E       assert 0 == 5
E        +  where 0 = clamp(-3, 0, 10)
============ 1 failed, 1 passed in 0.03s ============
```

Read it from the bottom up. The last line is the score. The `E` lines say what went wrong: `clamp` gave back `0` but the test expected 5. When a test fails, ask: **is the code wrong, or is my expected answer wrong?**

Normal and edge cases

A **normal case** is what usually happens. An **edge case** is a strange corner: that is where bugs love to hide.

Function	Normal case	Edge cases
`clamp(value, 0, 10)`	`clamp(5, 0, 10)` gives 5	too small, too big, exactly `0`, exactly `10`
`best_score(scores)`	best in the middle	empty list, one score, best is first, best is last
`encode(msg, shift)`	`"abc"` with shift 1	letters near z, spaces and `!`, shift `0`

Exercise 8.1: Test the Clamp

Folder: `exercises/week08/ex-test-the-clamp`. `starter.py` is **your test file**. Read `clamp_tools.py` first.

1. Press **Run**. How many tests say PASSED? _____ Why do the empty ones pass?

   ---

2. TODO 1 and TODO 2: write an `assert` line in each empty test. TODO 3: add a test for a value exactly on a limit.

3. Change one expected answer to something wrong. Run. Copy the last line of the output:

4. Change it back. Then run `check_my_work.py`: it tries your tests on three **broken** clamps. Catch them all!

Exercise 8.2: Test the Best Score

Folder: `exercises/week08/ex-test-best-score`. `best_score(scores)` finds the highest score. Plan your tests first:

I call...	I expect...	Normal or edge?
`best_score([3, 9, 2])`	9	normal
`best_score([])`		

Write the tests in `starter.py`, press **Run**, then check with `check_my_work.py`.

Exercise 8.3: Fix the Bugs the Tests Found

Folder: `exercises/week08/ex-fix-the-bugs`. Four game helpers, one bug each. The tests are already written.

1. Run `check_my_work.py`. Pick one `TRY AGAIN` and read its hint.
2. Find that test in `test_fix_the_bugs.py`. Read it like a sentence.
3. Fix **one** function in `starter.py`. Never change the test! Run the checks again. Repeat.

Fixed: ☐ `lives_after_hit` ☐ `medal` ☐ `name_tag` ☐ `average_score`

Tip: Why testing matters

In 1999 a spacecraft was lost on its way to Mars because two parts of its software used different units, and no test checked them together. People can only trust apps and games that have been tested. Testing your game is a way of being fair to your players.

Vocabulary box

test A small function that checks another function and says PASSED or FAILED.

assert "I am sure this is true." If it is false, Python stops with `AssertionError`.

pytest A helper that finds every function called `test_...`, runs it and reports the results.

normal case An everyday input, like a list of three scores.

edge case A strange corner: empty, zero, the first, the last, exactly on a limit.

expected vs actual What the test says should happen, and what really happened.

Reflection

Think of a game you like. Write one edge case its programmers should test.

When a test fails, what two things could be wrong?

Which was harder: writing tests or fixing bugs? Why?

Week 8 Level 2: Intermediate

Testing: Prove It Works

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Test the Cipher (about 10 minutes)

Folder: `exercises/week08/hw-test-the-cipher`. `encode()` in `cipher_tools.py` is the Caesar cipher from week 5, and it is correct. Your job: write tests so nobody can ever break it without you noticing. Keep your paper cipher wheel nearby.

1. Read `cipher_tools.py`. Then open `starter.py` (your test file) and press **Run**.
2. TODO 1: work out `encode("xyz", 3)` with the wheel and write a test for it: _____
3. TODO 2: test a message with a space and a `!` in it. My message and its secret version:

   _____
4. TODO 3: one more test of your own (ideas: shift `0`, or an empty message `""`).
5. Press **Run**: all PASSED? Then run `check_my_work.py`. It tries your tests on three broken ciphers.

★★ Challenge (want more?): Spot the edge

For each function, write one **normal** case and one **edge** case (no computer needed).

Function	Normal case	Edge case
`count_vowels(word)`: counts a, e, i, o, u		
`lives_left(lives, hits)`: never below `0`		
`last_item(items)`: the last item		

Creative (make it yours): Test something of yours

Pick a function you wrote in an earlier week (for example from your Monster Stats, Word Guess or Idea Notebook). In the same folder, make a new file called `my_tests.py`. Import your function (`from starter import my_function`), write two tests (one normal, one edge), and copy the last three lines of this week's `starter.py` to the bottom so that **Run** starts pytest. This is exactly how your final project's tests will work. Or: draw a comic strip of a "crash-test dummy" testing a game's strangest corner.

Self-check: tick each box when it is true

- ☐ My test names all start with `test_` and each has an `assert`.
- ☐ All my tests say PASSED on the correct `encode()`.
- ☐ `check_my_work.py` says my tests catch all three broken ciphers.
- ☐ I can explain the difference between a normal case and an edge case.